

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

Search the Help Center /

[Fireblocks Help Center](#)

[Backup & Recovery](#)

Backup and Recovery

Recover Canton Coin (CC)

Overview

This article describes how to recover Canton Coin (CC) as part of a validator disaster recovery procedure.

The process restores Canton Coin balances by re-hosting external parties on a new Canton validator and importing their Active Contract Set (ACS).

Before you begin, ensure you have:

- Your full recovery kit, including all required keys
- A new Canton validator node
- Console access to the node's participant pod or container

Important:

This procedure covers Canton Coin recovery only. Recovering tokens (e.g. USDCx) requires additional actions performed by the relevant Token Registrar (such as Circle).

Security notice

This procedure requires reconstruction of private key material. The security implications are identical to performing a standard workspace recovery.

Before proceeding:

- Review your organization's recovery security procedures.
- Perform signing operations on an offline machine whenever possible.
- Transfer data using secure methods only.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

1. Create a topology transaction.
2. Sign the transaction.
3. Publish the signed transaction.
4. Verify party-to-participant mapping.
5. Disconnect the participant.
6. Create a database snapshot.
7. Obtain the party Active Contract Set (ACS).
8. Import the ACS.
9. Reconnect the participant.
10. Clear the onboarding flag.

The official Canton reference documentation is available [here](#). Some commands below differ slightly from the official documentation to provide an additional safety margin.

Step 1: Create a topology transaction

After accessing the participant container console on the node, run:

```
// Replace YOUR_PARTY_ID with the external party ID
val partyId = PartyId.tryFromProtoPrimitive("YOUR_PARTY_ID")
val participantId = participant.id
val synchronizerId = participant.synchronizers.id_of("global")
val partyToParticipant = PartyToParticipant.tryCreate(
  partyId = partyId,
  threshold = PositiveInt.one,
  participants = Seq(
    HostingParticipant(
      participantId,
      ParticipantPermission.Confirming,
      onboarding = true
    )
  ),
)
import com.digitalasset.canton.admin.api.client.commands.TopologyAdminC
val topologyTransaction =
```

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

```
    )  
  )  
) .head  
topologyTransaction.hash.hash.toHexString
```

Record the generated transaction hash. You will sign the bytes represented by this hash in the next step. If recovering multiple parties, repeat this step for each party.

Step 2: Sign the transaction hash

1. Restore private key material using your Fireblocks Recovery Utility.
2. Use Raw Signing to sign the generated payload.

Requirements:

- Select **EDDSA** as the signing algorithm.
- Enter the relevant vault account index.
- Generate the signature and securely store it.

If Raw Signing is unavailable, upgrade the Recovery Utility. Repeat this step for each topology transaction created.

Step 3: Publish the signed transaction

Run the following command inside the participant console:

```
val signature =  
  Signature.fromExternalSigning(  
    SignatureFormat.Raw,  
    HexString.parseToByteString("HASH_SIGNATURE_HEXSTRING").get,  
    partyId.namespace.fingerprint,  
    SigningAlgorithmSpec.Ed25519  
  )  
val topologyTxSignedByParty =
```

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

```
),  
    isProposal = false,  
    ProtocolVersion.v34,  
  )  
  val topologyTxSignedByBoth =  
    participant.topology.transactions.sign(  
      topologyTxSignedByParty,  
      TopologyStoreId.Synchronizer(synchronizerId),  
    )  
  participant.topology.transactions.load(  
    topologyTxSignedByBoth,  
    TopologyStoreId.Synchronizer(synchronizerId),  
  )
```

For multiple parties, repeat using the corresponding signatures.

Step 4: Verify the party mapping

Confirm the **PartyToParticipant** mapping was updated:

```
participant.topology.party_to_participant_mappings.list(  
  synchronizerId,  
  filterParty = partyId.filterString  
)
```

Expected result:

- The party appears to be hosted on the participant.
- `onboarding = true` is shown.

To verify multiple parties:

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

You should see N + 1 parties:

- N recovered parties
- The participant's own party

Step 5: Disconnect the participant

Disconnect from all synchronizers:

```
participant.synchronizers.disconnect_all()
```

Disable automatic reconnection:

```
participant.synchronizers.modify(  
  "global",  
  _.copy(manualConnect = true)  
)
```



Repeat for additional synchronizers if applicable.

Step 6: Create a database snapshot

Create a snapshot of your validator database. Because implementations vary by database type, follow your database vendor's documentation.

Recommended best practice:

1. Stop validator and participant containers.
2. Create the snapshot.
3. Restart containers.
4. Reconnect to the console.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/



```
curl -sSL --fail-with-body \  
'https://scan.sv-1.global.canton.network.sync.global/api/scan/v0/acs/YOL  
-H 'Content-Type: application/json' \  
| jq -r .acs_snapshot | base64 -d > acs_snapshot
```

Replace:

- YOUR_PARTY_ID: target party ID
- YOUR_VALID_FROM: value from the mapping result in Step 4

For multiple parties, query each party separately.

Step 8: Import the ACS

Import the snapshot:

```
participant.repair.import_acs("<absolute_path_to_acs>")
```

For multiple imports:



```
Seq("<path1>", "<path2>", "...").map(  
p => participant.repair.import_acs(p)  
)
```

Important:

If this step fails, restore the database snapshot created earlier before retrying.

Step 9: Reconnect the participant

First record the current ledger position:

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

```
participant.synchronizers.reconnect_all()
```

Re-enable automatic reconnection:



```
participant.synchronizers.modify(  
  "global",  
  _.copy(manualConnect = false)  
)
```

Step 10: Clear the onboarding flag

Remove onboarding status so transactions can resume:



```
val (onboarded, minimalSafeClearingTs) =  
  participant.parties.clear_party_onboarding_flag(  
    partyId,  
    synchronizerId,  
    ledgerEnd  
  )
```

For multiple parties:



```
Seq(partyId1, partyId2).map(  
  party =>  
    participant.parties.clear_party_onboarding_flag(  
      party,  
      synchronizerId,  
      ledgerEnd  
    )  
)
```

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

Pre-approvals

Wallet providers must:

- Cancel previous pre-approvals configured for Fireblocks validators
- Create new pre-approvals if required

Contract IDs

Contract IDs change after migration. Wallet providers must retrieve updated contract IDs from their validator.

Token recovery

This procedure does not recover tokens. Token recovery requires coordination with the relevant Registrar (for example, Circle for USDCx).

Was this article helpful?

Yes

No

ON THIS PAGE

Overview

Security notice

Recovery workflow

Step 1: Create a topology transaction

Step 2: Sign the transaction hash

Step 3: Publish the signed transaction

Step 4: Verify the party mapping

Step 5: Disconnect the participant

Step 6: Create a database snapshot

Step 7: Obtain the party ACS

Step 8: Import the ACS

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

Help in this section

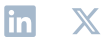
[fireblocks.com](#)

[Status](#)

[API Docs](#)

[Console](#)

[info@fireblocks.com](#)



Fireblocks © 2026. All Rights Reserved. NMLS Registration Number: 2066055

[Privacy Policy](#)